

Méthodes CIM et Actions d'Administration Système

Ce document constitue le support théorique et pratique avancé sur le pilotage du système d'exploitation Windows via les méthodes CIM (*Common Information Model*). Il détaille l'exploration des signatures de méthodes, le passage d'arguments structurés, l'exécution à distance via WinRM et le traitement rigoureux des codes de retour d'exécution.

1. Au-delà de l'Audit : Le Rôle des Méthodes CIM

La majorité des administrateurs système réduisent l'usage de CIM à la lecture d'indicateurs matériels ou système via Get-CimInstance. Cependant, le standard CIM est un modèle orienté objet complet combinant :

- Des **propriétés** : état statique ou compteurs de la machine (taille d'un disque, version du BIOS, charge CPU).
- Des **méthodes** : opérations exécutables capables de modifier la configuration, d'ordonner des actions bas niveau ou de piloter des composants sans interface graphique.

1.1 Comparatif : Modification via Cmdlets classiques vs Méthodes CIM

Critère d'évaluation	Cmdlets Windows dédiées (ex: Set-NetIPAddress, Restart-Computer)	Méthodes CIM (Invoke-CimMethod)
Disponibilité d'API	Dépendantes de modules spécifiques (parfois absents sous Windows Server 2012/2016 ou en environnement restreint).	Universelles et natives à 100 % sur tous les postes Windows via le service WMI/CIM (winmgmt).
Transport réseau	Varie selon le module (RPC dynamique, SMB, ou WinRM).	Standardisé et chiffré sur le port fixe 5985 (HTTP) ou 5986 (HTTPS).
Granularité d'action	Prédéfinie par les concepteurs de la cmdlet.	Accès direct aux API internes du pilote ou du sous-système Windows.
Gestion des retours	Objets d'erreur PowerShell	Codes numériques

	dans \$Error.	normalisés d'API (ReturnValue), requérant un contrôle explicite.
--	---------------	--

2. Découverte et Inspection des Méthodes (Get-CimClass)

Avant d'appeler une méthode, l'administrateur doit en connaître le nom exact, le mode d'appel (statique ou d'instance), ainsi que les paramètres d'entrée requis.

2.1 Lister les méthodes disponibles sur une classe

Pour cartographier les actions exposées par une classe, on interroge sa définition de schéma avec Get-CimClass :

```
# Découvrir les méthodes disponibles sur la gestion des adaptateurs réseau
(Get-CimClass -ClassName Win32_NetworkAdapterConfiguration).CimClassMethods

# Découvrir les méthodes d'extinction / redémarrage de l'OS
(Get-CimClass -ClassName Win32_OperatingSystem).CimClassMethods | Select-Object
-Property Name, ReturnType
```

2.2 Analyser les paramètres d'entrée et de sortie (InParameters / OutParameters)

Chaque méthode CIM dispose d'une signature formelle. Pour savoir quels arguments fournir à une méthode comme EnableStatic (définition d'une adresse IP) :

```
$ClasseNet = Get-CimClass -ClassName Win32_NetworkAdapterConfiguration
$MethodeIP = $ClasseNet.CimClassMethods["EnableStatic"]

# Inspection des paramètres d'entrée attendus :
$MethodeIP.Parameters | Format-Table -Property Name, CimType, Qualifiers
```

Résultat de l'analyse :

- **IPAddress** : tableau de chaînes (String[]).
- **SubnetMask** : tableau de chaînes (String[]).

3. Les Deux Modes d'Exécution : Statique vs Instance

Une méthode CIM s'exécute selon deux schémas fondamentaux, directement calqués sur la

programmation orientée objet :

- **[MODE 1 : MÉTHODE STATIQUE (Niveau Classe)]**
Invoke-CimMethod -ClassName Win32_Process -MethodName "Create"
 └─ Action globale ne dépendant pas d'un composant existant (Ex: Créer un processus).
- **[MODE 2 : MÉTHODE D'INSTANCE (Niveau Objet)]**
Get-CimInstance (Carte Réseau N°3) | Invoke-CimMethod -MethodName "EnableDHCP"
 └─ Action appliquée chirurgicalement sur un composant précis déjà instancié.

3.1 Mode Statique (Appel direct sur la classe)

On ne cible pas un objet déjà existant sur le système, mais la classe elle-même pour instancier ou créer une ressource :

```
# Création d'un processus distant (ex: lancer un script en arrière-plan)
Invoke-CimMethod -ClassName Win32_Process -MethodName "Create" -Arguments @{
    CommandLine = "powershell.exe -NoProfile -Command Start-Sleep -Seconds 30"
}
```

3.2 Mode Instance (Appel ciblé via le pipeline ou -InputObject)

L'action s'applique à un composant physique ou logique unique. Il est impératif de cibler d'abord l'instance exacte avec Get-CimInstance :

```
# 1. Sélection de la carte réseau active (ex: celle ayant IPEnabled = $true)
$CarteReseau = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter
"IPEnabled = True AND DHCPEnabled = False"
```

```
# 2. Exécution de la méthode de bascule DHCP sur cette instance précise
$CarteReseau | Invoke-CimMethod -MethodName "EnableDHCP"
```

4. Quatre Cas Pratiques d'Administration par Méthodes CIM

Cas Pratique 1 : Configuration Réseau Avancée (Bascule IP Statique)

Sur un serveur dont l'interface d'administration réseau classique échoue, CIM permet d'injecter une configuration IP de secours sans contact physique :

```
# 1. Isolement de la carte physique principale via WQL
$Carte = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter "Index = 1"

# 2. Définition des paramètres d'adressage IP (tableaux de chaînes)
```

```

$paramsIP = @{
    IPAddress = @"192.168.10.50"
    SubnetMask = @"255.255.255.0"
}

# 3. Application de l'adresse et du masque
$retourIP = $Carte | Invoke-CimMethod -MethodName "EnableStatic" -Arguments $paramsIP

# 4. Configuration de la passerelle par défaut (avec métrique de routage)
$paramsPasserelle = @{
    DefaultIPGateway = @"192.168.10.254"
    GatewayCostMetric = @[uint16]1
}
$retourGW = $Carte | Invoke-CimMethod -MethodName "SetGateways" -Arguments
$paramsPasserelle

# 5. Définition des serveurs DNS d'entreprise
$paramsDNS = @{
    DNSServerSearchOrder = @"192.168.10.10", "192.168.10.11"
}
$retourDNS = $Carte | Invoke-CimMethod -MethodName "SetDNSServerSearchOrder"
-Arguments $paramsDNS

```

Cas Pratique 2 : Gestion des Partages Réseau SMB (Win32_Share)

Pour provisionner un partage réseau local ou distant sans charger le module SMB dédié :

```

# Création d'un dossier et publication du partage SMB via méthode statique
$cheminPhysique = "C:\DepotLogs"

```

```

if (-not (Test-Path $cheminPhysique)) {
    New-Item -Path $cheminPhysique -ItemType Directory -Force | Out-Null
}

```

```

$argsPartage = @{
    Name      = "DepotLogs$"
    Path      = $cheminPhysique
    Type      = [uint32]0 # 0 = Disque standard
    MaximumAllowed = [uint32]10 # Limite de connexions simultanées
    Description = "Partage administratif d'audit"
}

```

```

$resultat = Invoke-CimMethod -ClassName Win32_Share -MethodName "Create" -Arguments

```

```
$ArgumentsPartage
```

```
if ($Resultat.ReturnValue -eq 0) {  
    Write-Host "Partage SMB créé avec succès." -ForegroundColor Green  
} else {  
    Write-Warning "Échec de création du partage. Code d'erreur CIM : $($Resultat.ReturnValue)"  
}
```

Cas Pratique 3 : Arrêt et Redémarrage Contrôlé (Win32Shutdown)

La cmdlet Restart-Computer peut parfois être bloquée par des applications récalcitrantes. La méthode Win32ShutdownTracker de Win32_OperatingSystem force l'action au niveau du noyau en consignait la raison de l'arrêt dans le journal d'événements Windows :

```
# Récupération de l'instance OS  
$OS = Get-CimInstance -ClassName Win32_OperatingSystem  
  
# Drapeaux d'extinction : 6 = Reboot forcé (2 = Reboot + 4 = Forcer la fermeture des applis)  
# Code raison : 0x40001 = Maintenance planifiée d'infrastructure  
$ArgsReboot = @{  
    Flags      = [sint32]6  
    Reserved   = [sint32]0  
    ReasonCode = [uint32]0x40001  
    Comment    = "Redémarrage automatisé post-déploiement de correctifs"  
}  
  
$OS | Invoke-CimMethod -MethodName "Win32ShutdownTracker" -Arguments $ArgsReboot
```

Cas Pratique 4 : Lancement d'un Processus avec Récupération de son PID

Contrairement à un simple appel d'exécutable, la méthode Create de Win32_Process retourne instantanément l'identifiant système (PID) de la tâche créée :

```
$ArgumentsProcessus = @{  
    CommandLine = "cmd.exe /c ping.exe -n 20 127.0.0.1 > C:\Temp\ping.log"  
}  
  
$Execution = Invoke-CimMethod -ClassName Win32_Process -MethodName "Create"  
-Arguments $ArgumentsProcessus  
  
if ($Execution.ReturnValue -eq 0) {
```

```

Write-Host "Processus lancé en arrière-plan avec le PID : $($Execution.ProcessId)"
-ForegroundColor Cyan
} else {
    Write-Error "Impossible d'instancier le processus. Code retour : $($Execution.ReturnValue)"
}

```

5. Déjouer le Piège des Erreurs Silencieuses (ReturnValue)

L'erreur la plus critique lors de l'utilisation de Invoke-CimMethod réside dans l'interprétation des échecs.

5.1 Pourquoi le bloc try / catch ne réagit pas ?

Par défaut, PowerShell considère qu'une commande a réussi dès lors que la trame réseau WS-Management a été acheminée et traitée par le démon distant.

- Si vous demandez à Win32_Share de créer un partage avec un chemin de dossier inexistant, la méthode CIM s'exécute sans lever d'exception PowerShell.
- Le bloc catch **ne se déclenche pas**.
- La commande retourne un objet contenant une propriété numérique : **ReturnValue**.

5.2 Tableau des codes de retour d'exécution usuels

Classe CIM	Code ReturnValue	Signification système
Toutes classes	0	Opération exécutée avec succès (Success).
Win32_NetworkAdapterCon figuration	1	Succès, mais un redémarrage machine est requis.
Win32_NetworkAdapterCon figuration	64	Adresse IP ou masque de sous-réseau non valide.
Win32_NetworkAdapterCon figuration	84	L'adresse IP n'est pas activée sur cet adaptateur.
Win32_Share	2	Accès refusé (privilèges insuffisants).
Win32_Share	9	Nom de partage non

		conforme ou invalide.
Win32_Share	22	Le partage existe déjà sur le système.
Win32_Process	2	Fichier exécutable introuvable.
Win32_Process	3	Droits insuffisants pour démarrer le binaire.

5.3 Patron de Validation Robuste en Production

Tout script d'administration qui appelle Invoke-CimMethod doit obligatoirement valider la propriété ReturnValue :

```
$Reponse = $Instance | Invoke-CimMethod -MethodName "MaMethode" -Arguments
$MonDictionnaire

if ($Reponse.ReturnValue -ne 0) {
    throw "L'action système a échoué avec le code d'erreur CIM : $($Reponse.ReturnValue)"
}
```

```
Write-Host "Action validée par le sous-système Windows." -ForegroundColor Green
```

6. Actions Multi-Machines avec Sessions Persistantes (CimSession)

Lorsque l'on doit appliquer des modifications en masse sur distants (ex: redémarrage coordonné ou reconfiguration DNS), ouvrir et fermer un socket WinRM pour chaque méthode génère une latence excessive. On associe alors une session persistante (CimSession) à Invoke-CimMethod :

```
$Serveurs = @("SRV-APP01", "SRV-APP02", "SRV-APP03")

foreach ($Cible in $Serveurs) {
    try {
        # 1. Ouverture du canal WinRM sécurisé
        $Session = New-CimSession -ComputerName $Cible -ErrorAction Stop

        # 2. Ciblage de la carte réseau principale sur la machine distante
        $Carte = Get-CimInstance -CimSession $Session -ClassName
```

```
Win32_NetworkAdapterConfiguration -Filter "IPEnabled = True AND DHCPEnabled = True"  
-ErrorAction Stop
```

```
if ($Carte) {  
    # 3. Exécution distante via la session  
    $Action = $Carte | Invoke-CimMethod -MethodName "RenewDHCPLease"  
  
    if ($Action.ReturnValue -eq 0) {  
        Write-Host "[${Cible}] Bail DHCP renouvelé avec succès." -ForegroundColor Green  
    } else {  
        Write-Warning "[${Cible}] Échec du renouvellement : Code $($Action.ReturnValue)"  
    }  
}  
} catch {  
    Write-Error "[${Cible}] Connexion WinRM impossible : $($_.Exception.Message)"  
} finally {  
    # 4. Libération systématique de la ressource réseau  
    if ($Session) {  
        Remove-CimSession -CimSession $Session  
    }  
}  
}
```

7. Synthèse des Bonnes Pratiques d'Administration par Méthodes CIM

1. **Vérifier systématiquement ReturnValue** : Ne jamais se fier au simple fait que la commande n'a pas affiché de texte rouge. Une méthode qui renvoie un code différent de 0 signale un échec fonctionnel.
2. **Insérer le typage exact dans la table de hachage** : Certaines méthodes attendent des entiers non signés stricts (ex: [uint32], [uint16]). Transtyper explicitement les valeurs dans le dictionnaire -Arguments.
3. **Privilégier le filtrage WQL en amont** : Pour les méthodes d'instance, isoler précisément l'élément cible via -Filter avant d'appeler la méthode afin d'éviter toute application accidentelle sur un composant non ciblé.
4. **Privilégier les CimSession pour les scripts multi-tâches** : Pour plusieurs actions consécutives sur une machine distante, utiliser une session persistante pour économiser la négociation d'authentification Kerberos/NTLM.